INTERVIEW PREPARATION SERIES

Stacks & Queues

LeetCode Practice Set — 10 Problems

This practice set covers **10 LeetCode problems** testing Stack & Queue concepts: LIFO matching, expression evaluation, monotonic stacks, BFS traversal, circular queues, and deque-based sliding windows. Each problem includes a 4-tier breakdown. Solve in any language — the concepts are universal.

PROBLEM 01

Valid Parentheses Easy

TIER 1 — UNDERSTAND

Given a string containing only `(`, `)`, `{`, `}`, `[`, `]`, determine if the input string is valid. Tests the fundamental stack LIFO property.

TIER 2 — APPROACH

Stack-based matching: push opening brackets, pop on closing brackets and verify the match.

TIER 3 — ALGORITHM SKETCH

```
def isValid(s):
    stack = []
    pairs = {')':'(', ']':'[', '}':'{'}
    for ch in s:
        if ch in '([{':
            stack.append(ch)
        elif ch in ')]}':
            if not stack or stack[-1] != pairs[ch]:
                return False
            stack.pop()
    return len(stack) == 0
```

TIER 4 — COMPLEXITY

Time: O(n) | **Space:** O(n)

Core concept: the most recently opened bracket must close first — pure LIFO.

PROBLEM 02

Evaluate Reverse Polish Notation Medium

TIER 1 — UNDERSTAND

Evaluate an arithmetic expression in Reverse Polish (postfix) Notation. Operators follow their operands: `["2","1","+","3","*"]` = 9. Tests stack-based expression evaluation.

TIER 2 — APPROACH

Operand stack: numbers go on the stack. Operators pop two operands, compute, push the result.

TIER 3 — ALGORITHM SKETCH

```
def evalRPN(tokens):
    stack = []
    ops = {'+': lambda a,b: a+b, '-': lambda a,b: a-b,
           '*': lambda a,b: a*b, '/': lambda a,b: int(a/b)}
    for t in tokens:
        if t in ops:
            b, a = stack.pop(), stack.pop()
            stack.append(ops[t](a, b))
        else:
            stack.append(int(t))
    return stack[0]
```

TIER 4 — COMPLEXITY

Time: O(n) | **Space:** O(n)

Core concept: stack-based evaluation is how calculators and bytecode VMs (JVM, CPython) process expressions internally.

PROBLEM 03

Daily Temperatures Medium

TIER 1 — UNDERSTAND

Given daily temperatures, return an array where each element is the number of days until a warmer temperature. If none, return 0. Tests the monotonic stack pattern.

TIER 2 — APPROACH

Monotonic decreasing stack: maintain a stack of indices whose temperatures have not yet found a warmer day. When a warmer day is found, pop all colder days and record the gap.

TIER 3 — ALGORITHM SKETCH

```
def dailyTemperatures(temperatures):
    n = len(temperatures)
    result = [0] * n
    stack = []                      # stack of indices
    for i in range(n):
        while stack and temperatures[i] > temperatures[stack[-1]]:
            prev = stack.pop()
            result[prev] = i - prev   # days until warmer
        stack.append(i)
    return result
```

TIER 4 — COMPLEXITY

Time: O(n) — each index pushed and popped at most once | **Space:** O(n)

Core concept: the monotonic stack processes elements that are "waiting" for a future event. The stack naturally backtracks when the future event arrives.

PROBLEM 04

Implement Queue using Stacks Easy

TIER 1 — UNDERSTAND

Implement a FIFO queue using only two stacks. Support `push`, `pop`, `peek`, `empty`. Tests deep understanding of both data structures.

TIER 2 — APPROACH

Lazy transfer: input stack for pushes, output stack for pops. Transfer all from input to output (reversing order) only when output is empty.

TIER 3 — ALGORITHM SKETCH

```
class MyQueue:
    def __init__(self):
        self.in_stack = []
        self.out_stack = []

    def push(self, x): self.in_stack.append(x)

    def _transfer(self):
        if not self.out_stack:
            while self.in_stack:
                self.out_stack.append(self.in_stack.pop())

    def pop(self):   self._transfer(); return self.out_stack.pop()
    def peek(self):  self._transfer(); return self.out_stack[-1]
```

```
    def empty(self): return not self.in_stack and not self.out_stack
```

TIER 4 — COMPLEXITY

Time: O(1) amortized for all operations | **Space:** O(n)

Core concept: two stacks invert each other's order. Transferring between them converts LIFO to FIFO.

PROBLEM 05

Implement Stack using Queues Easy

TIER 1 — UNDERSTAND

Implement a LIFO stack using only queues. Tests understanding of how FIFO and LIFO relate.

TIER 2 — APPROACH

Rotate on push: push the new element, then rotate the queue (dequeue and re-enqueue) n-1 times so the new element ends up at the front.

TIER 3 — ALGORITHM SKETCH

```
from collections import deque

class MyStack:
    def __init__(self): self.q = deque()

    def push(self, x):
        self.q.append(x)
        for _ in range(len(self.q) - 1):   # rotate
            self.q.append(self.q.popleft())

    def pop(self):  return self.q.popleft()
    def top(self):  return self.q[0]
    def empty(self): return not self.q
```

TIER 4 — COMPLEXITY

Time: O(n) push, O(1) pop | **Space:** O(n)

Core concept: rotating the queue after each push moves the newest element to the front, converting FIFO to LIFO.

PROBLEM 06

Number of Islands Medium

TIER 1 — UNDERSTAND

Given a 2D grid of '1' (land) and '0' (water), count the number of islands. An island is land connected horizontally/vertically. Tests BFS with a queue for graph traversal.

TIER 2 — APPROACH

BFS flood-fill: for each unvisited land cell, start a BFS. The queue explores all connected land cells, marking them visited. Each BFS call discovers one complete island.

TIER 3 — ALGORITHM SKETCH

```
def numIslands(grid):
    rows, cols = len(grid), len(grid[0])
    count = 0
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == '1':
                count += 1
                queue = deque([(r, c)])
                grid[r][c] = '0'     # mark visited
                while queue:
                    cr, cc = queue.popleft()
                    for dr, dc in [(0,1),(0,-1),(1,0),(-1,0)]:
                        nr, nc = cr+dr, cc+dc
                        if 0<=nr<rows and 0<=nc<cols and grid[nr][nc]=='1':
                            grid[nr][nc] = '0'
                            queue.append((nr, nc))
    return count
```

TIER 4 — COMPLEXITY

Time: O(m×n) | **Space:** O(min(m, n)) for queue

Core concept: BFS with a queue systematically explores all reachable cells level by level, ensuring no cell is visited twice.

PROBLEM 07

Rotting Oranges Medium

TIER 1 — UNDERSTAND

Rotten oranges infect adjacent fresh oranges every minute. Return the time until all oranges are rotten, or -1 if impossible. Tests multi-source BFS.

TIER 2 — APPROACH

Multi-source BFS: start by enqueuing ALL rotten oranges simultaneously. Each BFS level represents one minute.

TIER 3 — ALGORITHM SKETCH

```
def orangesRotting(grid):
    rows, cols = len(grid), len(grid[0])
    queue = deque()
    fresh = 0
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == 2: queue.append((r, c))
            elif grid[r][c] == 1: fresh += 1
    minutes = 0
    while queue and fresh > 0:
        minutes += 1
        for _ in range(len(queue)):      # process one level
            r, c = queue.popleft()
            for dr, dc in [(0,1),(0,-1),(1,0),(-1,0)]:
                nr, nc = r+dr, c+dc
                if 0<=nr<rows and 0<=nc<cols and grid[nr][nc]==1:
                    grid[nr][nc] = 2
                    fresh -= 1
                    queue.append((nr, nc))
    return minutes if fresh == 0 else -1
```

TIER 4 — COMPLEXITY

Time: O(m×n) | **Space:** O(m×n)

Core concept: multi-source BFS starts from all sources at once, simulating parallel spread. The queue's level-by-level processing naturally counts time steps.

PROBLEM 08

Design Circular Queue Medium

TIER 1 — UNDERSTAND

Design a circular queue with fixed capacity. Implement `enQueue`, `deQueue`, `Front`, `Rear`, `isEmpty`, `isFull`. Tests circular buffer implementation with modular arithmetic.

TIER 2 — APPROACH

Fixed array + front pointer + size counter: compute rear dynamically as `(front + size) % capacity`.

TIER 3 — ALGORITHM SKETCH

```
class MyCircularQueue:
    def __init__(self, k):
        self.data = [0] * k
        self.front = 0
        self.size = 0
        self.cap = k

    def enQueue(self, value):
        if self.isFull(): return False
        self.data[(self.front + self.size) % self.cap] = value
        self.size += 1
        return True

    def deQueue(self):
        if self.isEmpty(): return False
        self.front = (self.front + 1) % self.cap
        self.size -= 1
        return True

    def Front(self): return -1 if self.isEmpty() else self.data[self.front]
    def Rear(self):  return -1 if self.isEmpty() else
self.data[(self.front+self.size-1)%self.cap]
    def isEmpty(self): return self.size == 0
    def isFull(self):  return self.size == self.cap
```

TIER 4 — COMPLEXITY

Time: O(1) all operations | **Space:** O(k)

Core concept: modular arithmetic wraps indices around the array boundary. The size counter distinguishes full from empty.

PROBLEM 09

Design Circular Deque Medium

TIER 1 — UNDERSTAND

Design a double-ended circular queue supporting `insertFront`, `insertLast`, `deleteFront`, `deleteLast`. Extends the circular queue with operations at both ends.

TIER 2 — APPROACH

Same circular array as the queue, plus front-insertion: decrement front (with wrap-around) and write. Back-deletion: decrement size.

TIER 3 — ALGORITHM SKETCH

```
class MyCircularDeque:
    def __init__(self, k):
        self.data = [0]*k; self.front = 0; self.size = 0; self.cap = k

    def insertFront(self, value):
        if self.isFull(): return False
        self.front = (self.front - 1) % self.cap   # move front backward
        self.data[self.front] = value
        self.size += 1; return True

    def insertLast(self, value):
        if self.isFull(): return False
        self.data[(self.front + self.size) % self.cap] = value
        self.size += 1; return True

    def deleteFront(self):
        if self.isEmpty(): return False
        self.front = (self.front + 1) % self.cap
        self.size -= 1; return True

    def deleteLast(self):
        if self.isEmpty(): return False
        self.size -= 1; return True
```

TIER 4 — COMPLEXITY

Time: O(1) all operations | **Space:** O(k)

Core concept: the front pointer can move backward (wrap from 0 to capacity-1) to support front insertion, making the deque bidirectional.

PROBLEM 10

Sliding Window Maximum Hard

TIER 1 — UNDERSTAND

Given an array and window size k, return the maximum value in each sliding window position. Tests the monotonic deque pattern — the most advanced queue-based technique.

TIER 2 — APPROACH

Monotonic decreasing deque: maintain a deque of indices where values are in decreasing order. The front always holds the current window's maximum.

TIER 3 — ALGORITHM SKETCH

```
def maxSlidingWindow(nums, k):
```

```
    dq = deque()   # indices of decreasing values
    result = []
    for i in range(len(nums)):
        # Remove indices outside the window
        while dq and dq[0] < i - k + 1:
            dq.popleft()
        # Remove smaller elements from back
        while dq and nums[dq[-1]] <= nums[i]:
            dq.pop()
        dq.append(i)
        if i >= k - 1:
            result.append(nums[dq[0]])  # front = max
    return result
```

TIER 4 — COMPLEXITY

Time: O(n) — each element enters and leaves the deque at most once | **Space:** O(k)

Core concept: the deque removes from both ends — front for expired elements, back for smaller elements. This maintains a decreasing order invariant, making the maximum always accessible at O(1).

SUMMARY TABLE

#	Problem	Difficulty	Key Pattern	Topic
1	Valid Parentheses	Easy	Stack Matching	Stack: LIFO
2	Evaluate RPN	Medium	Operand Stack	Stack: Expression
3	Daily Temperatures	Medium	Monotonic Stack	Stack: Backtracking
4	Queue using Stacks	Easy	Lazy Transfer	Stack & Queue
5	Stack using Queues	Easy	Rotate on Push	Stack & Queue
6	Number of Islands	Medium	BFS Flood Fill	Queue: BFS
7	Rotting Oranges	Medium	Multi-Source BFS	Queue: BFS
8	Design Circular Queue	Medium	Ring Buffer	Circular Queue
9	Design Circular Deque	Medium	Bidirectional Ring	Circular Queue
10	Sliding Window Max	Hard	Monotonic Deque	Deque Advanced